DATABASE MANAGEMENT SYSTEMS · ASSIGNMENT 3

Database Design Analysis

Data redundancy, anomalies, functional dependencies and normalization

Project	FoodExpress – Online Food Delivery System	Submitted by	Dhruvan Karthik
DBMS	MySQL Community Server 8.0.46	GitHub	github.com/hidhruvan-prog
Database	`foodexpress`	Date	25 October 2026

1. Objective and Scope

This assignment examines the database design proposed earlier for **FoodExpress**, an online food delivery platform that connects customers, restaurants and delivery agents. The earlier design defined nine tables: `Customers`, `Addresses`, `Restaurants`, `MenuItems`, `DeliveryAgents`, `Orders`, `OrderItems`, `Payments` and `Reviews`. The analysis has four goals:

1. show how a poorly designed (un-normalized) version of the same data produces redundancy and insertion, deletion and update anomalies;
2. identify the functional dependencies (FDs) that hold between the attributes;
3. use those FDs to normalize the data step by step (1NF → 2NF → 3NF → BCNF) and check the proposed design against them;
4. list the remaining weaknesses in the proposed design and the improvements that produce the **finalized design** used in Assignments 5 to 9.

2. What a Poor Design Looks Like: the Single-Table Version

A common first attempt is to record everything about an order in one wide table, as a spreadsheet would. Every row is one dish inside one order, so all the facts about the customer, the restaurant, the rider and the payment are copied onto every dish row. The extract below uses six rows covering three orders.

order_id	cust_name	cust_phone	address	restaurant	rest_phone	item	price	qty	agent	pay_method	total
1001	Aarav Sharma	9845012345	Sobha Dream Acres, 560103	Spice Route Biryani House	08041234567	Chicken Dum Biryani	289	2	Ravi Kumar	UPI	797
1001	Aarav Sharma	9845012345	Sobha Dream Acres, 560103	Spice Route Biryani House	08041234567	Chicken 65	219	1	Ravi Kumar	UPI	797
1002	Ananya Iyer	9731198765	Jayanagar, 560011	Dosa Junction	08049876543	Masala Dosa	99	2	Imran Khan	Card	336
1002	Ananya Iyer	9731198765	Jayanagar, 560011	Dosa Junction	08049876543	Filter Coffee	49	2	Imran Khan	Card	336
1003	Aarav Sharma	9845012345	Embassy Tech Village, 560103	Spice Route Biryani House	08041234567	Chicken Dum Biryani	289	1	Ravi Kumar	Cash	528
1003	Aarav Sharma	9845099999	Embassy Tech Village, 560103	Spice Route Biryani House	08041234567	Chicken 65	239	1	Ravi Kumar	Cash	528

Table 1 – `FoodOrders`, a single flat table. Shaded cells show values that already contradict other rows (two phone numbers for one customer; a different price for the same dish, Chicken 65).

2.1 Data redundancy

- **Customer details** (name, phone, e-mail, password) are repeated on every dish of every order the customer places. A customer with 40 orders of 3 dishes each stores the same phone number 120 times.
- **Restaurant details** (name, phone, address, cuisine) are repeated for every dish ever sold by that restaurant.

- **Menu prices** are repeated each time a dish is ordered, so the table no longer has one authoritative price.
- **Order-level facts** (date, address, agent, payment method, total) are repeated once per dish in the order.
- **Derived values** such as `total` are stored even though they can be computed from price × quantity.

2.2 Anomalies caused by the single-table design

Anomaly	Example in FoodExpress	Why it happens
Insertion	A new restaurant, *Sweet Tooth Desserts*, joins the platform and publishes its menu, but it cannot be stored until somebody places an order, because `order_id` is part of the key and cannot be NULL. The same applies to a newly registered customer or a newly hired rider.	Independent facts (restaurant, menu, customer, rider) can only exist inside an order row.
Deletion	If order 1002 is deleted (for example, a test order is removed), the only record of *Dosa Junction*, its phone number, the price of *Filter Coffee*, customer *Ananya Iyer* and rider *Imran Khan* disappears with it.	Deleting one fact (an order) unintentionally deletes other, unrelated facts stored in the same row.
Update	Aarav changes phone number. Every row with his name must be updated; missing one leaves rows 5 and 6 disagreeing (Table 1). When the restaurant raises the price of a dish, old and new prices become mixed in the same column, so nobody can tell whether 289 or 239 is the true price.	One real-world fact is stored in many rows, so a change must be applied everywhere at once.

> **How poor design leads to anomalies.** All three anomalies have the same root cause: the table mixes several *entities* (customer, restaurant, menu item, rider, order, payment) in one relation, and its key (`order_id, item`) only determines some of the columns. Every column that depends on something smaller than the key, or on another non-key column, is repeated and can drift out of sync. Normalization removes exactly those dependencies.

3. Functional Dependencies

A functional dependency **X** → **Y** means that two rows that agree on X must also agree on Y. The FDs below come from the business rules of FoodExpress (for example, "an e-mail address belongs to exactly one customer" and "one review per order").

#	Functional dependency	Meaning
FD1	`customer_id → full_name, email, phone, password_hash, registration_date`	Customer details depend only on the customer.
FD2	`email → customer_id` `phone → customer_id`	E-mail and phone are unique, so both are candidate keys of Customers.
FD3	`address_id → customer_id, label, address_line, city, state, pincode, is_default`	A saved address belongs to one customer.
FD4	`pincode → city, state`	A postal code lies in one city and state (discussed in §5.3).
FD5	`restaurant_id → restaurant_name, cuisine_type, address, contact_phone, is_active`	Restaurant details.
FD6	`item_id → restaurant_id, item_name, description, price, category, is_available`	Menu item details; each item belongs to one restaurant.
FD7	`restaurant_id, item_name → item_id`	A restaurant does not list the same dish name twice.
FD8	`agent_id → agent_name, phone, vehicle_number, is_available`	Delivery agent details.

#	Functional dependency	Meaning
FD9	`order_id → customer_id, restaurant_id, address_id, agent_id, order_date, order_status`	Order header facts.
FD10	`order_id, item_id → quantity, price_at_order`	Line-level facts depend on the whole composite key.
FD11	`order_id → payment_id, amount, payment_method, payment_status, payment_date` `payment_id → order_id`	Exactly one payment per order (1 : 1).
FD12	`order_id → review_id, rating, comment, review_date` `review_id → order_id`	At most one review per order (1 : 1).
FD13	`order_id → total_amount`	Derived: total = Σ(quantity × price_at_order) + delivery fee.
FD14	`item_id → restaurant_id` and `address_id → customer_id`	Ownership dependencies that create cross-table rules (§5).

3.1 Classifying the dependencies in the flat table

The flat table's primary key is the composite `{order_id, item_id}`. Measured against that key:

- **Full dependency:** only `quantity` and `price_at_order` (FD10) need the whole key.
- **Partial dependencies** (violate 2NF): all order-header columns depend on `order_id` alone (FD9, FD11, FD12), and all menu columns depend on `item_id` alone (FD6).
- **Transitive dependencies** (violate 3NF): `order_id → customer_id → full_name, phone …`, `order_id → restaurant_id → restaurant_name …`, `order_id → agent_id → agent_name …`, and `order_id → address_id → address_line, city …`.

4. Normalization Step by Step

Normal form	Rule	What was done for FoodExpress	Resulting relations
UNF → 1NF	Atomic values; no repeating groups.	An order that lists several dishes in one cell is split into one row per dish. Composite fields are split: a full address becomes address_line, city, state and pincode.	FoodOrders(order_id, item_id, …all columns…)
1NF → 2NF	No non-key attribute depends on part of a composite key.	Columns that depend only on `order_id` move to Orders; columns that depend only on `item_id` move to MenuItems. Only quantity and price_at_order stay with the composite key.	Orders(order_id, …), MenuItems(item_id, …), OrderItems(order_id, item_id, quantity, price_at_order)
2NF → 3NF	No non-key attribute depends on another non-key attribute.	Transitive chains through customer_id, restaurant_id, agent_id and address_id are removed by giving each entity its own table and keeping only the foreign key in Orders. Payment and review facts, which have their own life cycle, move to their own 1 : 1 tables.	Customers, Addresses, Restaurants, DeliveryAgents, Payments, Reviews
3NF → BCNF	Every determinant is a candidate key.	Checked each table: the determinants are the primary keys plus the unique keys email, phone (Customers), phone, vehicle_number (DeliveryAgents), (restaurant_id, item_name) (MenuItems) and order_id (Payments, Reviews), all declared UNIQUE. The one exception, pincode → city, state, is discussed in §5.3.	Nine tables in BCNF, with one documented exception

This decomposition is **lossless**: every split is made on an attribute that is a key of one of the resulting tables (for example, Orders ⋈ OrderItems on `order_id`), so joining the tables rebuilds the original rows exactly. It is also **dependency-preserving**: each FD in Section 3 can be checked inside a single table, except the cross-table rules in FD14, which are enforced with a composite foreign key and a trigger (Assignment 6).

5. Analysis of the Proposed Nine-Table Design

The nine tables proposed earlier already match the 3NF decomposition above, so the large anomalies of Section 2 do not occur. A column-by-column review against the FDs, however, still finds the following redundancies and gaps.

#	Where	Dependency / problem	Resulting anomaly	Improvement
1	Reviews(customer_id, restaurant_id)	Both are already determined by `order_id` (FD9), so `review_id → order_id → customer_id` is a transitive dependency.	**Update/insert:** a review could name a different customer or restaurant than its own order, and nothing keeps them consistent.	Remove both columns. Reach the customer and restaurant through `order_id`.
2	Restaurants.avg_rating, DeliveryAgents.avg_rating	Derived data: the averages are computed from Reviews. The design also had no column that stores a rider's rating, so the agent average had no source data.	**Update:** every new, edited or deleted review must also update the stored average, or it becomes wrong.	Drop the stored averages. Add `agent_rating` to Reviews and compute averages in the view `v_restaurant_ratings` and in queries.
3	Orders.total_amount	Derived from OrderItems (FD13).	**Update:** adding or removing a line item without recomputing the total leaves the order inconsistent.	Store the only non-derivable part, `delivery_fee`, and compute subtotal and total in the view `v_order_totals`. Payments.amount is kept because it records what was actually charged.
4	Restaurants.address (one text field)	Not atomic: city and pincode are buried inside a string.	Queries such as "restaurants in Mumbai" need fragile LIKE '%Mumbai%' searches; spelling variants break them.	Split into `address_line`, `city` and `pincode` (1NF).
5	Orders(customer_id, address_id)	FD14: `address_id → customer_id`, so Orders stores customer_id twice (once directly and once through the address).	**Insert:** an order could be placed by customer 3 but delivered to customer 8's saved address.	Keep both (customer_id is the main access path) but add a **composite foreign key** (address_id, customer_id) → Addresses(address_id, customer_id) so the combination must exist.
6	Addresses used by old orders	Order history depends on the address row.	**Deletion/update:** deleting or editing a saved address would erase or rewrite where past orders were delivered.	Do not delete addresses that are in use (ON DELETE RESTRICT). Add `is_active` so a customer can hide an address without losing history.
7	Addresses.is_default	Business rule: at most one default address per customer.	**Insert/update:** a customer could end up with two default addresses.	Add a UNIQUE key on a generated column that holds customer_id only for the default row.
8	OrderItems(item_id) vs Orders(restaurant_id)	FD14: `item_id → restaurant_id` must match the order's restaurant.	**Insert:** a Dosa Junction dish could be added to a Napoli Slice order.	Enforce with a BEFORE INSERT trigger (a CHECK constraint in MySQL cannot read another table).
9	Missing delivery time	Orders recorded only `order_date`.	Delivery performance (minutes to deliver) could not be measured.	Add `delivered_at`, required exactly when status = 'Delivered' (CHECK).

5.1 Redundancy that is kept on purpose

`OrderItems.price_at_order` looks like a copy of `MenuItems.price`, but it is a different fact: the price *at the moment of ordering*. If it were removed, every price change would silently rewrite the value of past orders and invoices (an update anomaly on history). Keeping it is therefore correct normalization, not redundancy. Likewise `Payments.amount` records the amount actually charged, which can legitimately differ from a recomputed total (for example after a refund).

5.2 Why derived values are moved to views

Ratings and order totals are needed often, but storing them creates a second copy of information that already exists. MySQL views give the convenience without the anomaly: `v_order_totals` computes subtotal and total for every order, and `v_restaurant_ratings` computes review counts and average ratings. Their results are always consistent with the base tables.

5.3 Pincode → city, state

Strictly, FD4 makes `city` and `state` transitively dependent on `address_id` through `pincode`, so Addresses is in 3NF only if a separate `Pincodes(pincode, city, state)` table is added. FoodExpress keeps the address exactly as the customer types it, because the platform does not maintain an official pincode directory and a missing directory entry must not block an order. This is a documented, deliberate exception. A `CHECK` constraint still makes sure each pincode is a valid six-digit Indian PIN.

6. Proposed Improvements – Before and After

Table	Proposed design	Finalized design
Customers	full_name, email, phone, password_hash, registration_date	Unchanged. UNIQUE (email), UNIQUE (phone), CHECK on phone and e-mail format.
Addresses	address_line, city, state, pincode, is_default	+ label (Home/Work/Other), + is_active (soft delete), one default per customer, UNIQUE (address_id, customer_id) as the target of the composite FK.
Restaurants	address (text), avg_rating	address → address_line, city, pincode. avg_rating removed (now in a view).
MenuItems	item_name, description, price, category, is_available	+ is_veg. UNIQUE (restaurant_id, item_name). CHECK price > 0 and category list.
DeliveryAgents	agent_name, phone, vehicle_number, is_available, avg_rating	avg_rating removed. + city (service area), + joined_on. UNIQUE phone, UNIQUE vehicle_number.
Orders	…, order_status, total_amount	total_amount → delivery_fee (total in view). + delivered_at. Composite FK (address_id, customer_id).
OrderItems	order_id, item_id, quantity, price_at_order	UNIQUE (order_id, item_id). CHECK quantity 1–50. Trigger: item must belong to the order's restaurant.
Payments	order_id, amount, method, status, date	Unchanged structure. UNIQUE (order_id), CHECK amount ≥ 0.
Reviews	order_id, customer_id, restaurant_id, rating, comment	customer_id and restaurant_id removed. rating → food_rating, + agent_rating. Trigger: only delivered orders can be reviewed.

7. Finalized Entity–Relationship Diagram

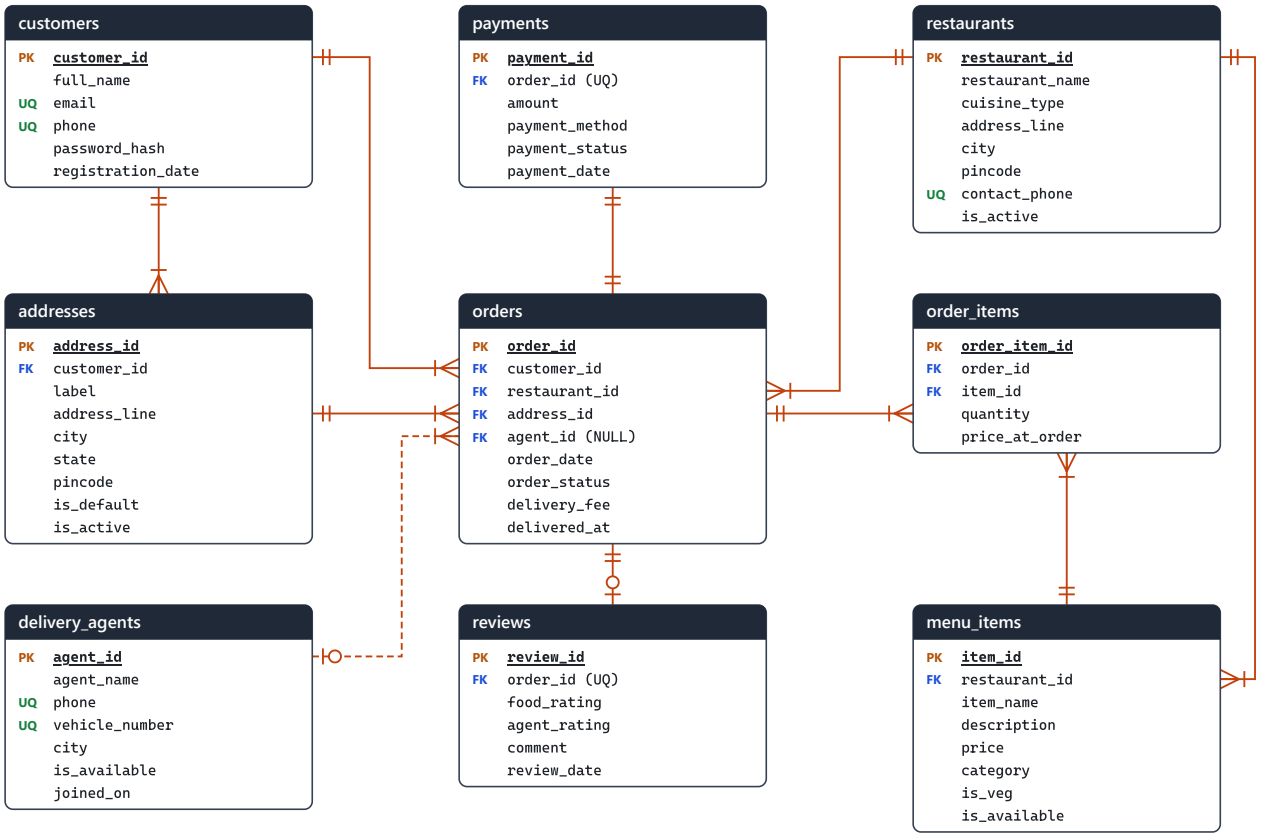


Figure 1 – Finalized FoodExpress schema (9 tables, crow's-foot notation). This design is implemented in Assignment 5.

PK primary key **FK** foreign key **UQ** unique — one-to-many: bar (one) to crow's foot (many) ○ optional (zero or one) dashed: nullable foreign key

8. Conclusion

A single-table design for food delivery stores each customer, restaurant, price and rider many times. That redundancy causes insertion anomalies (a restaurant cannot exist without an order), deletion anomalies (removing an order erases a restaurant) and update anomalies (one phone number in many rows). Identifying the functional dependencies and normalizing to 3NF/BCNF yields the nine-table structure proposed earlier. A closer review of that structure found remaining transitive dependencies (Reviews), stored derived values (ratings and totals), a non-atomic address, and cross-table rules that were not enforced. The finalized design fixes each of these with schema changes, views, a composite foreign key and triggers, while deliberately keeping `price_at_order` as a historical snapshot.